Area and Power efficient Floating Point Multiplier using Radix-4 Booth Encoding

K.V.Gowreesrinivas
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
srinivas.ece@anits.edu.in

R.Sindhuja
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
raghupatrunisindhuja.22.ece@.edu.in

G.Geethanjali
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
guntamukkalageethanjali.22.ece@anits.edu.in

Ch Murali Krishna
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
chittirimuralikrishna.22ce@anits.edu.in

Ch Sri Haranadh
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
chodisettisriharanadh.22.ece@anits.edu.in

Chukka Anoosha
Department of ECE
Anil Neerukonda Institute of Technology and Sciences
Visakhapatnam, India
anoosha.ece@anits.edu.in

2026 1st International Conference on VLSI Circuits and Systems (ICVCS) | 979-8-3195-3071-4/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICVCS69586.2026.11683186

Abstract*—Multiplication of floating-point data is a basic operation in digital signal processing, scientific computing, and embedded systems. The design of the mantissa multiplication block is a significant factor in the area, delay, and power consumption of a floating-point multiplier (FPM) based on the IEEE-754 standard. Initially, a 24x24 mantissa multiplier is designed by utilizing different adders. Among those adders, the combination of Carry Save Adder (CSA) and Radix-4 Booth encoding is found to have better performance in terms of area, speed, and power consumption due to the reduced number of partial products and absence of carry propagation delay. The optimized mantissa multiplier is then integrated with sign and exponent processing units to form a complete IEEE-754 (SPFPM). The design is described by using a structural Verilog HDL and is implemented by utilizing the Xilinx Vivado tool. The functionality of the proposed architecture is validated by performing simulations and is also validated by implementing the design on FPGA platform. In addition, area and power analysis are also carried out by utilizing the Cadence synthesis tool for evaluating the performance of the design in an ASIC environment. The proposed design is implemented on Nexys 4 DDR FPGA and realized on 45nm ASIC platforms. The experimental study shows that the proposed multiplier has utilized hardware (115 slice LUTs) and consumed 5720.574 µm2 of area and 272.47 µW of power, which is less as compared to existing FPMs.

Keywords: Digital signal processing, Floating point multiplier, Radix-4 booth encoder.

I. INTRODUCTION

Multipliers plays a vital role in high-performance digital design. They are used in FIR filters, microprocessors, and digital signal processors. However, they act as a bottleneck in the performance of digital systems. This is because they operate at a slower speed and consume more area. To overcome this problem, different architectures of multipliers are introduced with balanced constraints. Radix 2^n multipliers operate at high speed by processing digits in parallel rather than bits. This allows digit-level pipelining. This technique was introduced by M. K. Ibrahim[1]. This technique produces a modular and iterative design. The speed is almost constant in all cases, irrespective of the size of the multiplier.

Floating-point operations are at the centre of signal processing, image processing, machine learning accelerators, and various scientific simulations[2]. To make the multiplier more efficient in terms of speed and power dissipation, high-speed and low-power multipliers incorporate techniques such as the Booth algorithm[3]. This reduces the complexity and increases the speed. Power dissipation is one of the major factors to be considered in VLSI design. Dynamic power dissipation is the primary contributor to power dissipation. This is caused by switching activity in the circuit. This can be minimized by the expression $Pd = \alpha CLV^2f$.

The IEEE-754 single precision format is a common choice as it is a reasonable compromise between precision and hardware cost[4]. The mantissa multiplication operation dominates the critical path as well as switching activity. The array multipliers require a lot of partial products. Recent research focuses on optimizing IEEE-754 FPM to enhance Digital Signal Processing (DSP) and FPGA performance. Advanced techniques such as Vedic mathematics, Karatsuba-Ofman multiplication, and Canonic Signed Digit (CSD) encoding combined with Parallel Prefix Adders have been explored to address trade-offs between area, delay, and power consumption. Experimental results using Xilinx tools demonstrate that such hybrid architectures outperform conventional IP cores in terms of speed and hardware efficiency, making them suitable for real-time applications[5]-[12].

The use of radix-4 Booth encoding reduces the number of partial products by half, i.e., N/2, by creating overlapping groups of three bits. This method is more area-efficient and has fewer logic levels when compared to simple shifting and addition multiplication, while still providing an exact arithmetic operation.

The paper proposes a structural IEEE-754 FPM with a radix-4 Booth-encoded 24x24 mantissa multiplier. This paper makes the following contributions:

979-8-3195-3071-4/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.

- Structural IEEE-754 **SPFPM** implementation.
- Optimised 24×24 Radix-4 Booth mantissa multiplier.
- Reduced partial product reduction depth.
- FPGA implementation without DSP block usage.
- Improved area and power efficiency with exact arithmetic accuracy.

A. IEEE 754 FLOATING POINT MULTIPLICATION

A single precision IEEE-754 floating point number is represented as:

$$A = (-1)^{S_A} \times (1.M_A) \times 2^{E_A - Bias}$$

$$B = (-1)^{S_B} \times (1.M_B) \times 2^{E_B - Bias}$$

Where S is the sign bit, E is the exponent, M is the mantissa and Bias value is 127.

The product is computed as:

$$P = (-1)^{S_A \oplus S_B} \times (M_A \times M_B) \times 2^{(E_A \times E_B - Bias)}$$

Multiplication involves: Sign computation, exponent addition, 24×24 mantissa multiplication, normalization and rounding.

II . *PROPOSED ARCHITECTURE*

The IEEE-754 SPFPM is designed at a fully structural level, which means that each arithmetic operation is directly implemented at a logic level. The entire design is divided into three major steps: first is the extraction and preprocessing; second is the mantissa multiplication operation using Radix-4 Booth encoding; and last is the post-processing operation that includes normalisation and rounding.

A. Architecture Overview

In IEEE-754 single precision, a floating-point number is a 32-bit number consisting of a sign bit, eight exponent bits, and twenty-three mantissa bits. The mantissa is a normalised fraction, so a leading one is implicitly added before multiplication, resulting in a 24-bit significand.

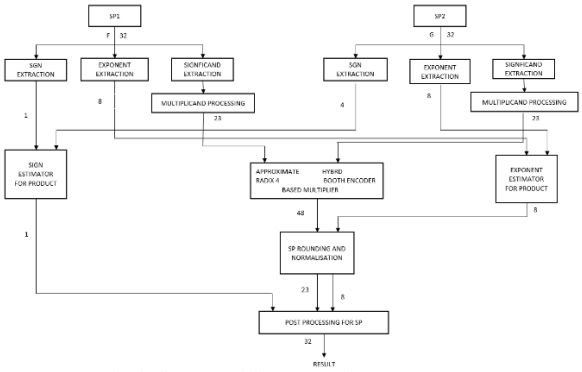


Fig:1 Block diagram of the proposed IEEE-754 SPFPM[5].

The architecture described in fig(1) is synchronous. In every clock cycle, two 32-bit floating-point numbers are processed to obtain a product result that is 32 bits. The datapath has parallel sign handling, exponent adder, and a 24x24 mantissa multiplier. Among these operations, the mantissa multiplication is the one that has the greatest amount of processing and hence needs to be optimised. The fig(1) is taken reference from[13].

B . Sign and Exponent Processing Unit

The sign of the result is determined by a simple exclusive OR operation on the signs of the operands. Mathematically, it is represented as follows:

$$SP = S_A \oplus S_B$$

This operation introduces virtually no delay and does not affect the critical path of the system. For the exponent, we need to perform the following operation:

$$E_P = E_A + E_B - 127$$

*C. Implementation of 24*24 Mantissa multiplier*

The mantissa multiplier has been implemented using different types of adders like Carry lookahead adder, Carry Save adder, Ripple Carry adder, Carry Save adder and Carry Skip Adder respectively. The combination of CSA and Radix –4 booth multiplier gives better result which is shown in fig (2) which is designed by taking reference from[14] where they implemented for 8 bit..

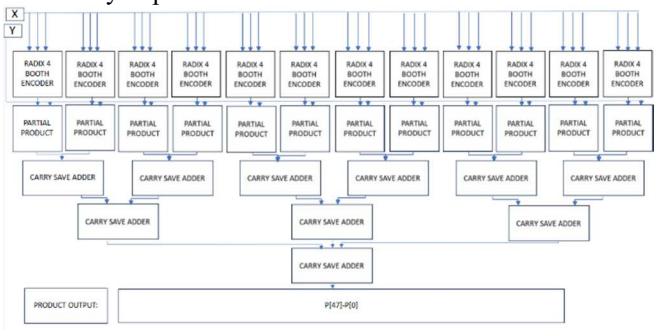


Fig:2 Block diagram for implementation of 24 bit mantissa multiplier using CSA and radix-4 booth multiplier

Carry Save Adder

A Carry Save adder or CSA shown in fig.3 is a digital adder circuit, and it is mostly used for finding the sum of a minimum of three or above binary numbers in a very efficient manner. A CSA is usually employed in a binary multiplier, as in this type of multiplier, the addition of the above two binary numbers is performed. This method helps in creating a big adder, which is much faster compared to the usual addition of numbers addition.

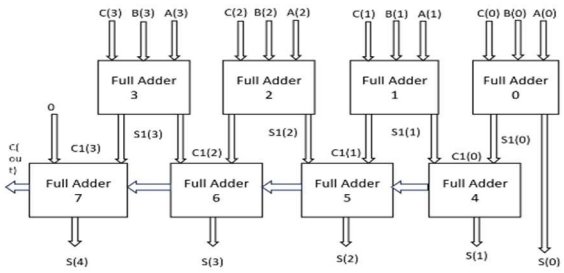


Fig:3 Basic Structure of Carry Save adder

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.

Radix -4 Booth Encoder Mantissa multiplier

Booth encoding using Radix – 4 is a clever variation on the standard multiplication technique. By redefining the multiplier in a higher base form, it greatly reduces the number of partial products to be computed during a signed binary multiplication. In a standard shift-and-add multiplication scheme, an N-bit multiplier produces N partial products, one for each bit. This increases hardware complexity, logic depth, and switching activity. This multiplication technique overcomes this problem by encoding two bits from the multiplier at a time, or equivalently, performing two bits in one multiplication.

It reduces the number of partial products by considering the multiplier digits in sets of three. For a multiplier value Y, the digits are taken as (y2i+1, y2i, y2i−1). Here, the digit y2i−1 is the previous digit, ensuring that the signed recoding is performed correctly. These sets of three digits overlap to ensure that the transition from one digit to the next is handled properly. This helps generate both positive and negative multiples.

Both groups choose a signed multiple of the multiplicand M. Depending on the bit pattern, the partial product can be 0, +M, -M, +2M, or -2M as shown in table(1). For a 24-bit mantissa multiplication, it results in only 12 partial products instead of 24 in a standard array multiplication.

Table 1 : Radix-4 Booth Encoding Table

Block	Partial Product
000	0
001	M
010	M
011	2M
100	-2M
101	-M
110	-M
111	0

The negative partial products are generated using two's complement logic. By generating ±2M through left shifting, additional multiplication hardware is avoided, thereby improving area efficiency.

Partial Product Generation

Each encoded group produces one signed partial product with correct shifting to represent its bit position. With Radix-4 encoding, you obtain half the number of partial products, and this reduces switching activity and interconnect significantly.

The partial products are generated concurrently through structural logic to ensure equal propagation delays and a regular hardware structure. This regularity helps improve efficiency in mapping to FPGAs.

Partial Product Reduction

Finally, after obtaining these partial products, there are 12 in number, and they all have to be added to produce a 48-bit result. Instead of sequentially summing them, a structured carry-save adder reduction tree is used.

The CSA accepts three inputs and produce two outputs, namely sum and carry, without carry propagation. This reduces the depth of the addition network. This process is repeated until two operands are left, and these two operands are added with a carry-propagate adder to produce the result. This reduces the carry propagation delay and prevents the formation of long carry chains, thus improving the performance.

Table 2: Comparison table for implementing mantissa multiplier using different adders.

Parameter	RCA	CLA	CSLA	CSKA	CSA
Power (W)	79.182	108.914	110.811	87.886	66.196
Slice LUTs	1534	2021	2089	1670	987
Slice	449	602	617	501	270
LUT as logic	1534	2021	2089	1670	987
Bonded IOB	96	96	96	96	96
Data Path Delay (ns)	48.486	46.641	48.379	46.539	16.133

The Carry Save Adder (CSA) performs better than all other adders in all parameters. It provides a power consumption that is 40.2% lower than the worst-case designs, an area reduction of 52.7%, and a speed improvement of 66.7%. The reason for its high efficiency is the avoidance of carry-propagation and low switching activity. In addition, the integration of CSA and Radix-4 Booth Multiplier produces the optimized result for high-performance VLSI systems in terms of power, area, and speed.

D. *Normalization and Rounding*

The result of the 24x24 mantissa multiplication is 48 bits. Given that both inputs are normalised, we can assume that the result has either a 1 in the most significant position or requires shifting left to normalize.

If we have a 1 in the most significant position, we increment our exponent. Otherwise, we shift left and adjust our exponent.

After the mantissa multiplication, the 48-bit product is obtained, and the rounding and normalization[3] operations are performed. The product is split into two parts, the upper 24 bits[15] and the lower 24 bits[16]. The lower 24 bits are used to derive the GRS bits, which indicate whether the rounding is performed. Once we have rounded, we can remove our hidden leading 1 and arrive at our final 23-bit mantissa.

Finally, the sign, exponent, and mantissa are concatenated to form the 32-bit IEEE-754 product.

III. RESULTS AND ANALYSIS

The SPFPM was implemented and verified using Xilinx Vivado Design Suite, starting with the mantissa multiplier and extending to the complete 32-bit design. Simulation results confirmed correct functionality, and synthesis analysis

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.

evaluated area, power, and timing performance. The design was further validated on the Nexys 4 DDR Digilent FPGA Board based on the Artix-7 FPGA. The results demonstrate efficient hardware utilization with reliable real-time performance.

*A. Implementing 24*24-bit Mantissa multiplier*

The design includes a CSA module and a 24×24 mantissa multiplier as the top module. The mantissa multiplier includes Booth encoding, partial product generation, and a multi-level CSA tree for reduction. Finally, a carry propagate adder produces the final product, which achieves high-speed multiplication with reduced delay.

Mantissa Multiplier Functional Simulation

To verify our design is correct for the 24×24 Radix-4 Booth mantissa multiplier, functional simulations were carried out using Vivado. Several operand pairs were used to ensure that partial product generation is correct, accumulation is correct, and the result is correct.

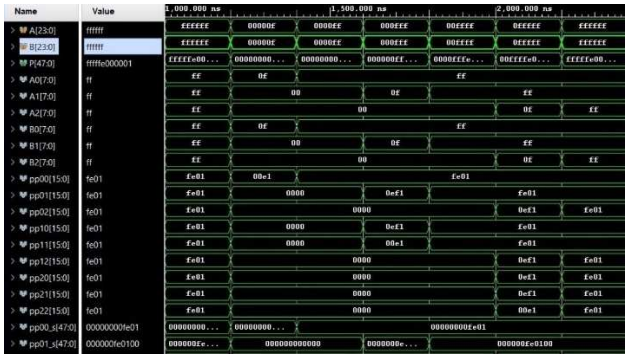


Fig:4 Functional simulation waveform of 24×24 mantissa multiplier.

The waveform shown in fig.4 shows the correct multiplication results for different input combinations. The simulation validates the structural implementation of the Radix-4 Booth encoding and accumulation logic.

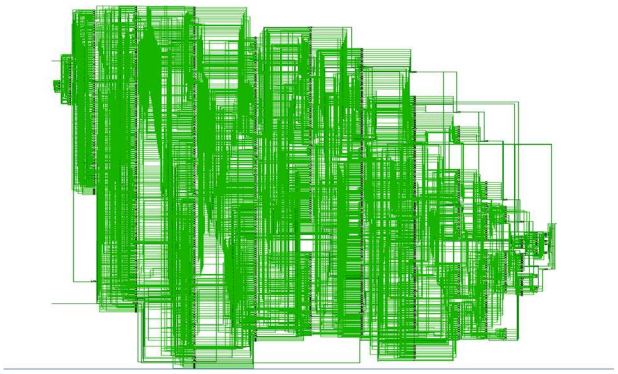


Fig:5 Schematic results for implementing a 24*24 mantissa multiplier.

Implementing final SPFPM

The design includes a top module named fp-mult_32 that has three major sub-modules, namely, input interface, floating-point multiplier core, and display driver. The multiplier core also includes sign, exponent, and mantissa units, as well as multiplication and normalization units. The final output is obtained through a multiplexed 7-segment decoder.

Overall Floating Point Multiplier Simulation

The complete IEEE-754 behavioural simulation. The simulation validates sign computation, exponent addition, mantissa multiplication, normalisation, and rounding stages.

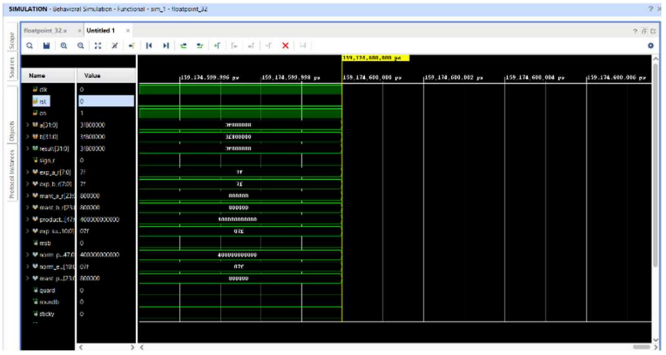


Fig:6 Functional simulation waveform of IEEE-754 SPFPM.

The waveform shown in fig(6) indicates that the partial products are computed correctly and the results of multiplication align properly. It also verifies that the Radix-4 Booth encoding and accumulation logic have been implemented correctly.

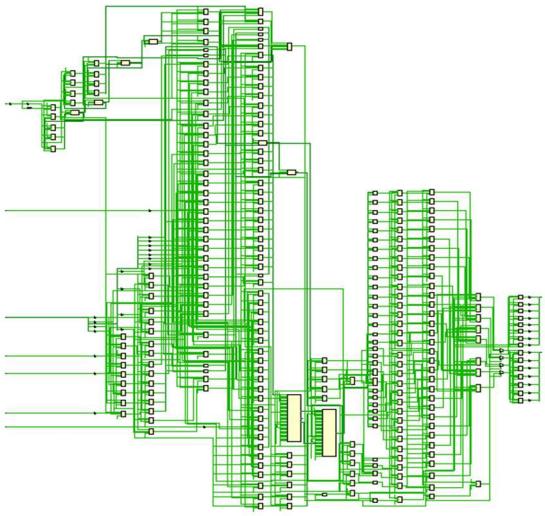


Fig:7 Schematic results of SPFPM.

B. Simulation Results for SPFPM

After integrating all these functional blocks, such as sign calculation, exponent management, multiplication of 24×24 Radix-4 Booth algorithm-based mantissa multiplication, normalisation, and rounding, the entire IEEE-754 SPFPM was synthesized and analysed. The final implementation result is given in Table 3.

Table 3: Final results for the implementation of an SPFPM.

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.

Parameter	Sub-Parameter	FP multiplier 32 bit
Power (W)	Default power =125°C	27.573
Slice LUTs	multiplier_24_booth_struct	115
Slice	multiplier_24_booth_struct	39
LUT as logic	multiplier_24_booth_struct	115
Bonded IOB	multiplier_24_booth_struct	96
Source (input - port)	report_timing -max_paths-sort by slack	B[28]
Destination (output - port)	report_timing -max_paths-sort by slack	result[28]
Data Path Delay (ns)	report_timing -max_paths-sort by slack	30.637ns
Logic level	report_timing -max_paths-sort by slack	20

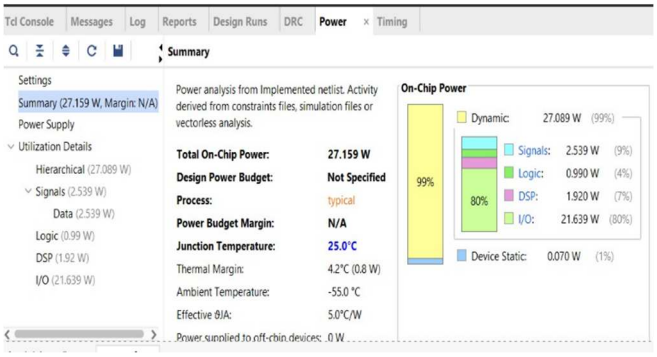


Fig:8 Shows the power analysis of SPFPM

The synthesis indicates that the complete 32-bit FPM achieves a data path delay of 30.637 ns and a logic depth of 20 levels. The multiplier utilises 115 LUTs and 39 slices, without any dedicated DSP block usage. Under default conditions, the total power consumption is 27.573 W is shown in fig(8). From the synthesis results, it is evident that the proposed structural Radix-4 Booth-based architecture is efficient in terms of area and timing requirements and is suitable for FPGA realisation. Power reduced to almost 70%.

D.FPGA Implementation

Final SPFPM is implemented on FPGA.Radix-4 Booth encoder, partial product generation, carry-save tree, and final carry-propagate addition, were implemented in LUT-based logic, and there was no usage of DSP slices The proposed IEEE-754 single-precision floating-point multiplier was described in structural Verilog HDL and was implemented after synthesising it with the Xilinx Vivado tool. The target technology library was Xilinx FPGA.

A clock constraint was used during the synthesis phase to achieve an estimate of the maximum possible frequency. After the place and route phase, static timing analysis was conducted. It was found that the critical path delay was determined in the multiplication process of the mantissa. Hence, optimization of the partial product reduction tree and the adder stage was conducted.

The SPFPM is implemented on the Nexys 4 DDR FPGA shown in fig(9). The specifications for the Nexys 4 DDR digilent FPGA board is xc7a100tcsg324-1, and the package is csg324.

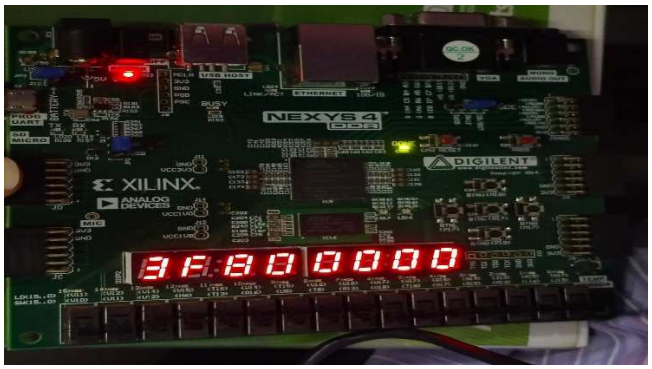


Fig:9 Implementation of SPFPM on Artrix 7 FPGA Board

IV. CONCLUSION

The proposed floating-point multiplier provides better area efficiency and reduced critical-path delay.

It was implemented as a whole using LUT-based structural logic and was mapped onto an FPGA. The results shows that the proposed floating-point multiplier is efficient in terms of resource usage, operating frequency, and power consumption, while maintaining IEEE-754 floating-point precision.

This is a good choice for implementing floating-point operations in FPGAs, where many scientific and signal processing applications are performed.

Table 4: Comparison of area and power of proposed with conventional SP FP multiplier, HFPM and existing multiplier unit.

Design	Area(μm²)	Power(μW)
Conventional Radix-8	8008	1262.36
Conventional Radix-4	9191	1381.71
Jiang et al [15]	7408	1198.39
Liu et al. [17]	7125	1168.45
Leon et al. [18]	6487	1028.57
Fu et al. [19]	6344	1005.16
Kuo and Wu [20]	6208	986.54
Hybrid Floating Point[5]	6110	948.31
Proposed	5720.574	272.47

Table 5: Hardware utilization of FP multipliers and the proposed FPM implemented on FPGA platform.

Design	Slice LUTs
Conventional Radix-8	920
Conventional Radix-4	965
Jiang et al [15]	902
Liu et al. [17]	944
Leon et al. [18]	911
Fu et al. [19]	885
Kuo and Wu [20]	849
Hybrid Floating Point [5]	831
Proposed	115

The proposed FP multiplier has shown significant improvement with an area reduction of about 6.3%, power consumption reduced by 71.3%, and LUT utilization reduced by 86.1% compared to the Hybrid FP approach. In conclusion, it can be stated that the proposed approach based on CSA and

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.

Radix-4 Booth encoding yields an efficient design for high-performance VLSI systems.

REFERENCES

[1] Aggoun, A. & Farwan, A.F. & Ibrahim, M.K. & Ashur, Abdall. (2005). Radix-2n serial-serial multipliers. Circuits, Devices and Systems, IEE Proceedings -. 151. 503 - 509. 10.1049/ip-cds:20040412.

[2] M.K. Jaiswal, R.C. Cheung, Area-efficient architectures for double precision multiplier on FPGA, with run-time-reconfigurable dual single precision support, Microelectron. J. 44 (5) (2013) 421–430.

[3] A.D. Booth, A signed binary multiplication technique, Q. J. Mech. Appl. Math. 4 (2) (1951) 236–240.

[4] IEEE standard for floating-point arithmetic, IEEE Standard 754-2008 (2008) 1–70.

[5] K. V. Gowreesrinivas and P. Samundiswary, "Design and analysis of single precision floating point multiplication using Karatsuba algorithm and parallel prefix adders," *2017 Fourth International Conference on Signal Processing, Communication and Networking (ICSCN)*, Chennai, India, 2017, pp. 1-6, doi: 10.1109/ICSCN.2017.8085729.

[6] K. V. Gowreesrinivas and P. Samundiswary, "Implementation and Analysis of Single Precision Floating Point Multiplication Using Vedic and Canonic Signed Digit Algorithm," *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, Bengaluru, India, 2018, pp. 1-4, doi: 10.1109/ICCCNT.2018.8494184.

IEEE standard for floating-point arithmetic, IEEE Standard 754-2008 (2008) 1–70.

[7] U. Singh, R. Mundliya, R. Maharana and B. Jajodia, "Floating Point Multipliers on FPGAs," *2025 Devices for Integrated Circuit (DevIC)*, Kalyani, India, 2025, pp. 702-707, doi: 10.1109/DevIC63749.2025.11012142.

[8] M. H. Yacoub, S. M. Ismail and L. A. Said, "Hardware Realization of High-Speed Area-Efficient floating point Arithmetic Unit on FPGA," *2024 International Conference on Machine Intelligence and Smart Innovation (ICMISI)*, Alexandria, Egypt, 2024, pp. 190-193, doi: 10.1109/ICMISI61517.2024.10580214.

[9] N. Chabini, M. C. Wolf and R. Beguenane, "LUT-Based Multipliers for IEEE-754 FP Arithmetic on FPGAs," *2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, Yorktown Heights, NY, USA, 2024, pp. 698-701, doi: 10.1109/UEMCON62879.2024.10754781.

[10] H. Zhang, X. Zheng and S. -B. Ko, "High Performance and Energy Efficient floating point Multiplier on FPGA," *2023 IEEE 3rd International Conference on Computer Systems (ICCS)*, Qingdao, China, 2023, pp. 178-182, doi: 10.1109/ICCS59700.2023.10335508.

[11] C. Rakshitha and M. Roopa, "Low Latency based Floating Point Multiplier using Parallel Prefix Adders," *2025 6th International Conference on Electronics and Sustainable Communication Systems (ICESC)*, Coimbatore, India, 2025, pp. 274-280, doi: 10.1109/ICESC65114.2025.11212524.

[12] S. Rahaman, K. C, K. Nunna, J. D and M. N. Chaithanya, "Floating Point Multipliers Based On Vedic Mathematics," *2023 International Conference on Network, Multimedia and Information Technology (NMITCON)*, Bengaluru, India, 2023, pp. 1-7, doi: 10.1109/NMITCON58196.2023.10275996.

[13] Pranose J. Edavoor, Aswini K. Samantaray, Amol D. Rahulkar,Design of floatingpoint multiplier using approximate hybrid Radix-4/ Radix-8 booth encoder for image analysis, e-Prime - Advances in Electrical Engineering, Electronics and Energy, Volume 8, 2024,

[14] Dr.T. Esther Rani, Design of high performance Configurable Radix-4 Booth Multiplier Using Cadence Tools, CVR Journal of Science and Technology, Volume 6, June 2014

[15] H. Jiang, J. Han, F. Qiao, F. Lombardi, Approximate radix-8 booth multipliers for low-power and high-performance operation, IEEE Trans. Comput. 65 (8) (2016)2638–2644

[16] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, J. Cong, Optimizing fpga-based accelerator design for deep convolutional neural networks. Proceedings of the 2015 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, 2015, pp. 161–170.

[17] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, F. Lombardi, Design of approximate radix-4 booth multipliers for error-tolerant computing, IEEE Trans. Comput. 66 (8) (2017) 1435–1441.

[18] V. Leon, G. Zervakis, D. Soudris, K. Pekmestzi, Approximate hybrid high radix encoding for energy-efficient inexact multipliers, IEEE Trans. Very Large Scale Integrat. (VLSI) Syst. 26 (3) (2018) 421–430.

[19] C. Fu, X. Zhu, K. Huang, Z. Gu, An 8-bit radix-4 non-volitile parallel multiplier, Electronics 10 (2021) 2358.

[20] C.-T. Kuo, Y.C. Wu, FPGA implementation of a novel multifunction modulo (2n ±1) multiplier using radix-4 booth encoding scheme, Appl. Sci. 13 (2023) 10407.

Authorized licensed use limited to: LAHORE UNIV OF MANAGEMENT SCIENCES. Downloaded on November 03,2026 at 07:35:41 UTC from IEEE Xplore. Restrictions apply.